

HW 3 - Sampling-based motion planning & inspection planning

Algorithmic robot motion planning - 236767

Liraz Calif

Id: 211765391

2 2D: A* , RRT and Inspection Planning

2.1 A* Implementation

3. Try out different values of ε to see how the behavior changes. **Report** the final cost of the path and the number of states expanded for $\varepsilon = 1, 10, 20$.

For Map 1:

ε	<i>numer of expanded states</i>	<i>cost of final path</i>
$\varepsilon = 1$	56	17.66
$\varepsilon = 10$	54	25.07
$\varepsilon = 20$	48	25.07

For Map 2:

ε	<i>numer of expanded states</i>	<i>cost of final path</i>
$\varepsilon = 1$	12944	298.9
$\varepsilon = 10$	481	338.1
$\varepsilon = 20$	443	337.5

4. **Discuss** the effect of ε on the solution quality.

Answer:

We can see that ε affects the quality of the solution. For smaller values of ε , the planner produces lower path costs on both maps. This is expected because increasing ε gives more weight to the heuristic relative to the actual path cost. Our heuristic uses the Euclidean distance to the goal, which is admissible. Therefore, when $\varepsilon = 1$, the algorithm corresponds to the standard A*, which guarantees an optimal path.

When ε increases, the planner becomes greedier toward the goal. It tends to follow a more direct path toward the goal, avoiding side branches until necessary. This explains why fewer nodes are expanded and why the path cost can increase if obstacles force a detour. Meaning, weighted A* trades off exploration for directness toward the goal, which is better in open environments but can be problematic in tight corridors. For standard A* with $\varepsilon = 1$, the path is optimal, but the number of expanded nodes is the highest. When ε is increased, fewer nodes are expanded, the planner explores fewer side branches, and the computation time decreases.

In map 1, which is a small environment where the robot occupies a relatively large area, increasing ε from 1 to 20 decreases the number of expanded nodes by about 16 percent. However, the path cost increases substantially, rising about

40 percent when ε goes from 1 to 10. Further increasing ε from 10 to 20 does not significantly change the path cost, while the number of expanded nodes decreases by an additional 11 percent. These results suggest that in small and constrained environments, achieving an optimal path may be more important than reducing computation time, it may be preferable to use standard A*.

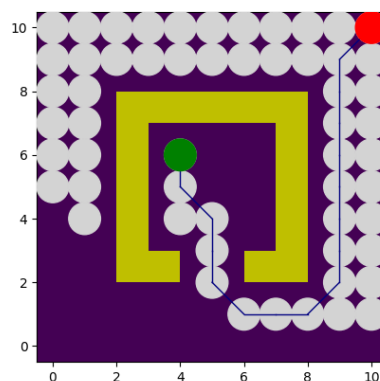
In map 2, which is a large environment where the robot is small relative to the map size, increasing ε from 1 to 10 increases the path cost by only about 13 percent, while the number of expanded nodes decreases dramatically, roughly by a factor of 30. Increasing ε further from 10 to 20 slightly improves the path cost and further reduces the number of expanded nodes. From these observations, we can understand that in larger and more open environments, using weighted A* can substantially reduce computation time, while the increase in path cost remains small and acceptable.

In conclusion, there is a clear tradeoff between computation time and path cost. The choice of ε should therefore consider the environment. In tight environments relative to the robot size, path optimality is more important than computation time, so standard A* is better. In large or open environments, weighted A* with higher ε can substantially reduce computation time while keeping the path cost acceptable. Also, when using weighted A*, increasing ε , for example from 10 to 20, usually does not significantly increase the path cost but can still slightly reduce the number of expanded nodes, providing a small additional computational gain.

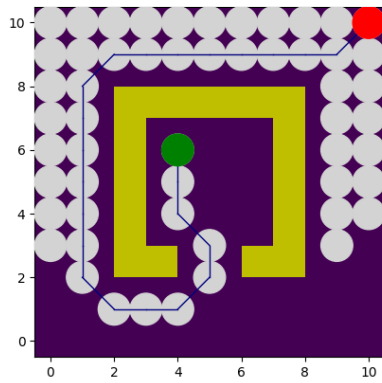
5. **Visualize** the final path in each case and the states visited (notice that the visualization automatically draw the states that were visited by extracting `expanded_nodes` so make sure to fill this list).

Map 1:

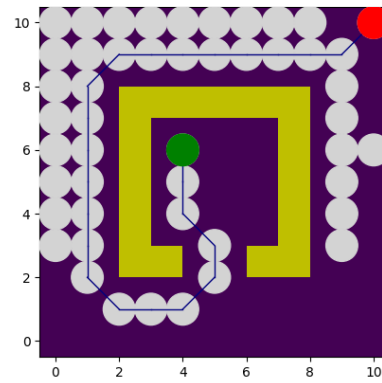
$$\varepsilon = 1$$



$\epsilon = 10$

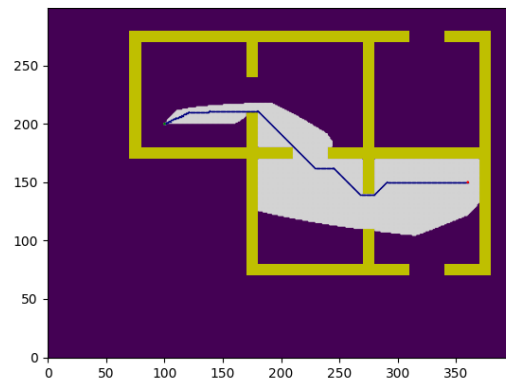


$\epsilon = 20$

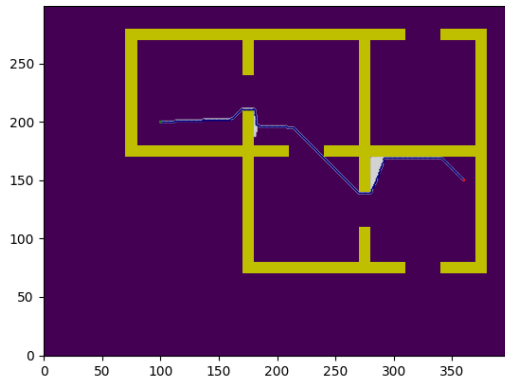


Map 2:

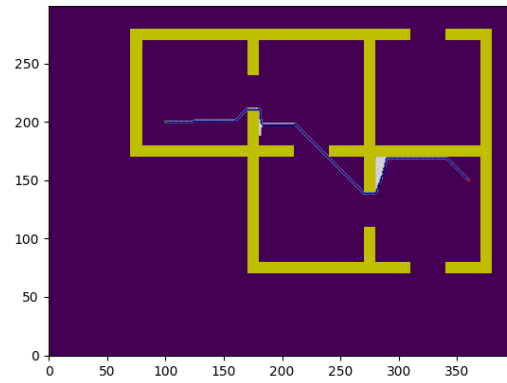
$\epsilon = 1$



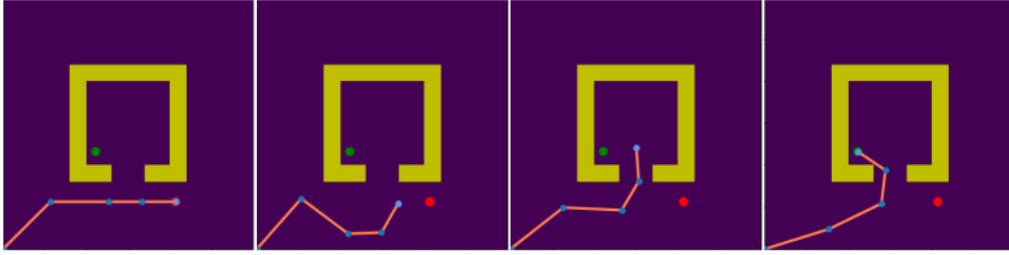
$\epsilon = 10$



$\epsilon = 20$



2.2 Motion Planning (25 points)



In the previous assignment, you implemented the basic building blocks for implementing sampling-based motion planning algorithms on the 2D manipulator environment. Here, we will use these to implement the single-query algorithms. You will start by implementing the RRT algorithm: at first, use the `DotEnvironment` (as in previous section) for debugging and then run and **report** results for the 2D robotic manipulator. In the file `RRTMotionPlanner.py`, implement the following functions:

1. **extend** - You will implement two versions of the extend function:

E1 extending an edge from the nearest neighbor in the RRT tree all the way to the sampled configuration.

E2 extending an edge from the nearest neighbor in the RRT tree by a step-size η towards the sampled configuration. Pick a small η of your choice and mention it in your write-up.

For each version of the extend function **report** the performance (cost, time) **for the 2D manipulator environment** and include a figure showing the final state of the tree for both biasing parameters **for the Dot environment**. Which extend strategy would you employ in practice?

2. **compute_cost** - Compute the cost of the given path, as the sum of the distances of all steps.
3. **plan** - The body of your planning algorithm with goal biasing. Make sure that the output is a numpy array of the calculated path in the configuration space. This array should (i) include the start and target configuration and (ii) be of dimension $N \times 4$ for N configurations.

Here, we would like to understand how fast the algorithm can find a path using different goal biasing and what is the quality of this path. For goal biasing of 5% and 20%, run 10 different executions of your algorithm until a solution is found when running over the 2D manipulator environment.

Report the average time required to find a solution and its average cost. Also include stdev of these values. In addition, **plot** the outcomes of all twenty runs on a graph where the x -axis is the time and the y -axis is the cost (use a different color for the 5% and 20% biases).

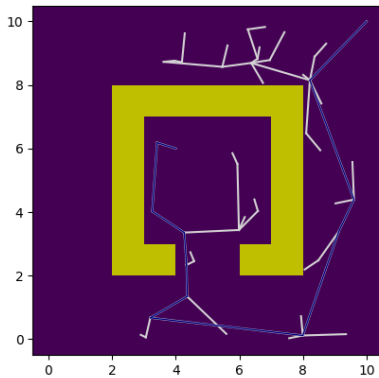
Finally, attach to the submission folder the visualizations created by `visualize_plan` for one representative instance for each parameter combination.

1. Part 1 - E_1 Vs E_2

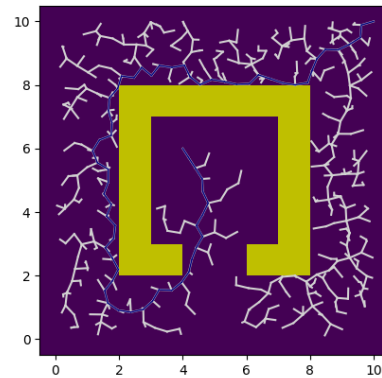
We will compute both extend functions for the 2D manipulator environment:

			$bias = 5\%$
E_1	<i>Time</i>	<i>run 1</i>	0.877
		<i>run 2</i>	1.456
	<i>Cost</i>	<i>run 1</i>	4.46
		<i>run 2</i>	6.85
E_2	<i>Time</i>	<i>run 1</i>	0.169
		<i>run 2</i>	0.789
	<i>Cost</i>	<i>run 1</i>	3.94
		<i>run 2</i>	3.59

For E_1 :



For E_2 :



After looking at the results of both practices, E_1, E_2 we can see that by using E_2 practice we got on both runs a smaller path cost and smaller time of planning.

By limiting η – the max step size we can take, we force the graph to expand evenly in space. E_2 explores broadly across the environment, still the controlled step size prevents overly large jumps and produces a more consistent coverage of the space. In contrast, E_1 extends randomly with varying step sizes, which can result in uneven exploration, unnecessary detours, and higher path costs.

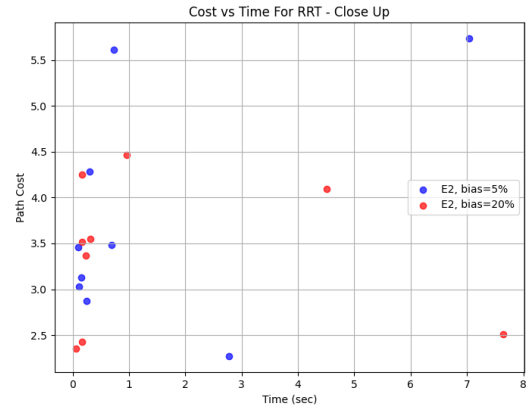
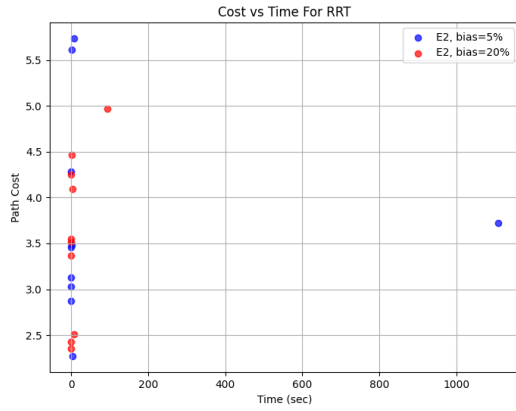
In addition, while E_1 attempts larger jumps to cover distance quickly, these often result in collision checks that fail in cluttered environments. The controlled expansion of E_2 reduces the wasted computations, leading to a faster convergence to the goal.

In conclusion, E_2 tends to produce a tighter path toward the goal, lowering the overall path cost, while also reducing planning time.

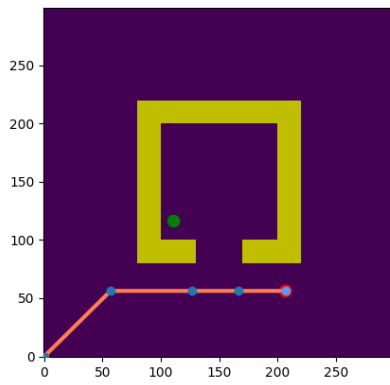
2. Part 2 – goal biasing

From now on we will use E_2 :

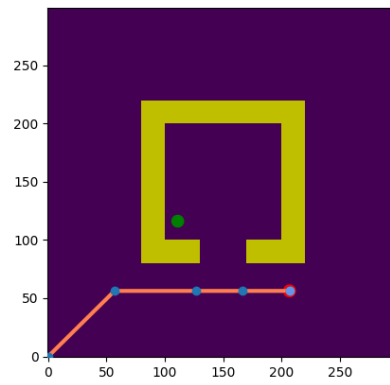
			<i>bias</i> = 5%	<i>bias</i> = 20%
E_2	<i>Time</i>	<i>Mean</i>	111.98	10.76
		<i>Stdev</i>	331.88	27.63
	<i>Cost</i>	<i>Mean</i>	3.76	3.55
		<i>Stdev</i>	1.083	0.862



For $p_{bias} = 5\%$



For $p_{bias} = 20\%$

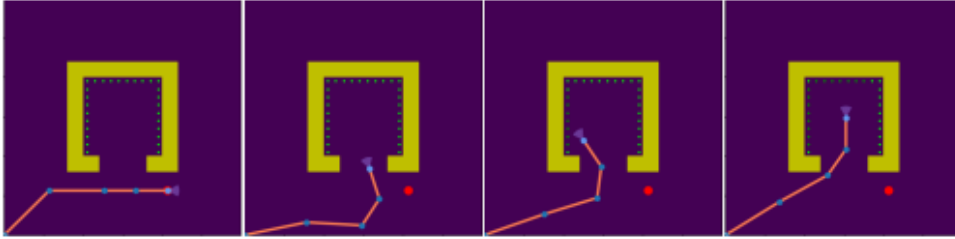


I attached them, they are called:

plan_rrt_2d_20_percent

plan_rrt_2d_5_percent

2.3 Inspection Planning (25 points)



In inspection planning, we are required to inspect a set of points of interest (POI) and not reach a predefined goal. In this exercise you are required to adapt your RRT implementation to handle the inspection task (this is slightly easier than the IRIS algorithm that will be taught in class). A vertex v_i in the RRT tree, will now also include the subset of POI that were seen so far when following the path from the root of the RRT tree to the configuration associated with the vertex and set $I_i = \{p_1, \dots, p_i\}$ to be this subset of POI. Namely, if in the (vanilla) motion-planning problem we defined a vertex as $v_i := [\theta_{i,1}, \theta_{i,2}, \theta_{i,3}, \theta_{i,4}]$, it will now be $v_i := [\theta_{i,1}, \theta_{i,2}, \theta_{i,3}, \theta_{i,4}, I_i]$. Moreover, if the edge (v_{i-1}, v_i) exists, then the set of POI inspected must be $I_i = I_{i-1} \cup S(v_i)$, where $S(v_i)$ is the set of inspection points that the robot can see at step vertex v_i .

Next, we define the *coverage* of a vertex v_i as $\frac{|I_i|}{|I_{\max}|}$, where $I_{\max}$ is the set of all POI. Namely, a vertex with coverage of 1.0 corresponds to a path along which all POI have been inspected.

To this end, the stopping condition of your planner will be reaching not a goal vertex but a vertex with a desired coverage and return the path to it. Implement the following in `RRTInspectionPlanning.py`:

1. `extend` and `compute_cost` - These can stay the same as in the motion-planning task.
2. `compute_union_of_points` (in `building_blocks.py`) - Compute the union of two sets of inspection points.
3. `plan` - The body of your planning algorithm. Make sure that the output is a numpy array of the calculated path in the configuration space (should be a shape of $(N, 4)$ for N configurations, including start and end configurations).

Report your performance (cost of the path and execution time) for coverage of 0.5 and 0.75, averaged over 10 executions for each, and attach the visualizations created by `visualize_plan` for one representative instance for each parameter combination.

Discuss the changes you were required to do in order to compute inspection planning. Notice that for

higher coverage the search may take a few minutes, so think about how you can improve your search! Make sure it is as efficient as possible.

Hint: In motion planning we biased the sampling towards a goal vertex. If there is no goal, how can we bias the sampling to improve coverage?

In order to bias for inspection planning we tried different methods. The changes that are required for inspection sampling are:

First instead of reaching the goal, we check for coverage, only when the node path has reached sufficient coverage along the way, the algorithm terminates and send the path back. Second, we needed to change the "goal" bias. In order

to reach the correct coverage, we biased from the point that saw the most inspection points towards the closest unseen point, by some distance factor. In order to do that we needed to use inverse kinematics after selecting the point. We implemented a stuck method, in which if the method gets stuck, for lower coverage, we are adding random points to make the tree larger, for higher coverage rates, if we get stuck, we are trying to move from max coverage node in the tree to the farthest unseen point for that node. Our method worked for coverage of 50%, but if looking at higher coverage, it stuck sometimes. Meaning our goal picking, or stuck goal picking method was not sufficient enough to reach higher coverage.

Coverage			
0.5	Average time	123.16	73.18
	Average cost	6.11	1.79

The representee visualize is:

rrt_50_coverage

3 RRT*

3.1 2D RRT* (15 points)

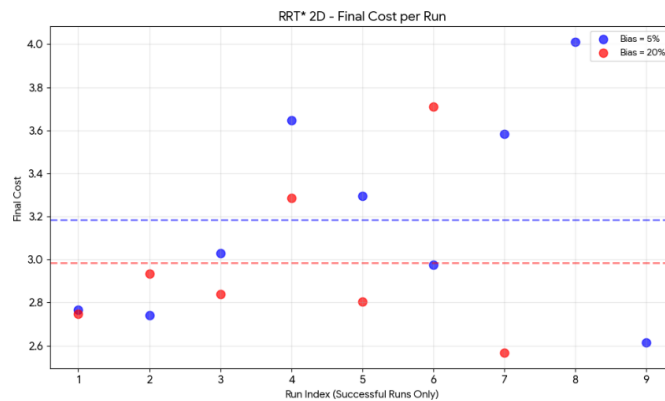
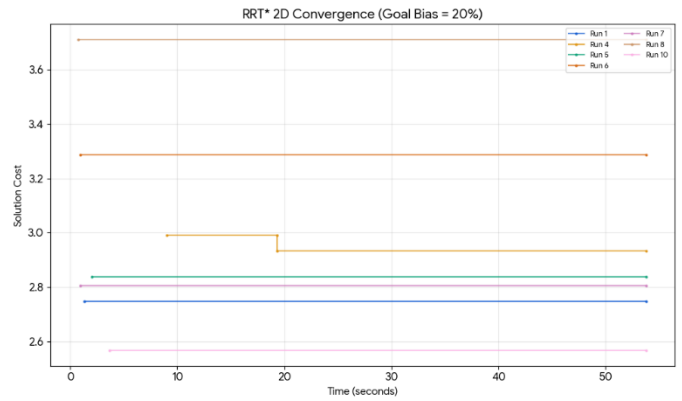
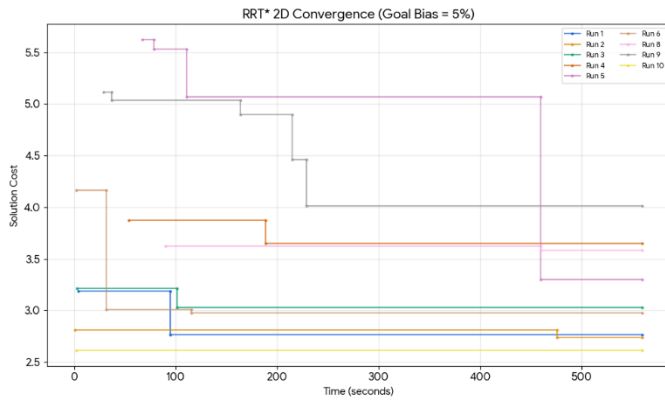
Now we would like you to implement the RRT* algorithm for the 2D manipulator environment. Please fill in the logic in `RRTStarPlanner.py`

Here, we would like to show the cost of the solution obtained by your algorithm as a function of time (use a max timeout of five times the average timeout you reported in Sec. 2.2) for different goal biases and different random seeds (as the algorithm is probabilistic by nature).

Specifically, submit two plots (x -axis is the running time and y -axis is the solution cost), one using goal biasing of 5% and one using 20%, each including ten algorithm runs. For each run, log when your algorithm improves the quality of its solution and the cost of the solution. Then, each algorithm run will be plotted as a line connecting these values.

Finally, use the function `visualize_plan` for one representative instance for each parameter combination and submit this visualization.

Discuss the differences by comparing the algorithm to A* and RRT. Is one run sufficient enough for evaluating such algorithms? You are encouraged to run more different start and goal configurations.



Looking at the different algorithm there are a few differences in a few point:

Path Cost:

A* is a deterministic algorithm, therefore using standard A* guarantee to find the shortest path if one exists. With Weighted A*, the path cost may be higher than optimal, but the number of expanded nodes decreases, which reduces computation time.

For standard RRT, the path cost is usually higher because the algorithm focuses on feasibility rather than optimality. Increasing goal bias to 20% we saw decreases in the average cost by steering the tree more directly toward the goal.

RRT*, on the other hand, produces lower average costs than standard RRT due to rewire method. Whenever a new node is added, RRT* checks whether it can provide a shorter path to nearby nodes and updates the tree accordingly. This results in smoother, more efficient paths. Like with RRT, higher goal bias (e.g., 0.2) tends to reduce the path cost.

Time:

Higher goal bias reduces the time to find an initial solution for both RRT and RRT*. However, RRT* requires more time than standard RRT because the rewire operation is computationally expensive. Even with a timeout five times longer

than the average RRT time, some RRT* instances may fail to find a path. This happens because the algorithm may expand into irrelevant areas due to its probabilistic nature or spend significant time rewiring before reaching the goal. We can see that for RRT too, in some instances the algorithm takes more time due to the randomness. In contrast, A* systematically expands nodes toward the goal, making it more predictable and efficient in computation.

Problem complexity:

RRT and RRT* are useful for systems with higher degrees of freedom, such as robotic arms. This is because A* become computationally infeasible as the state space grows exponentially. Sampling-based algorithms like RRT and RRT* scale better with high-dimensional spaces, providing feasible paths even when exact optimality is difficult to compute.

Number of Runs:

For A*, one run is sufficient to evaluate the algorithm because it is deterministic; the same start, goal, and parameters will always produce the same path and cost.

For RRT and RRT*, one run is insufficient due to their probabilistic nature. Random sampling causes performance, time, and cost to vary across runs, even with identical parameters. To evaluate these algorithms properly, a large number of runs is needed to calculate meaningful statistics, such as mean path cost, success rate, and standard deviation.

The visualized plans are called:

rrt_star_2d_5_percent

rrt_star_2d_20_percent

3.2 3D RRT* (15 points)

We will now continue to the 3D environment and run the RRT* algorithm on it. You will study the effects of the parameters on the algorithm's performance. Then, we will visualize the paths in an advanced simulation and at the lab.

1. In this lab, there is the added constraint of ensuring that the robot does not hit the window. One solution could have been to model the wall as an obstacle approximated by a set of spheres. This is a bad idea (why?). Instead, add a condition to the function `config.validity_checker()` to return `False` if the manipulator exceeds the plain 0.4 [m] in x -direction. Provide the function the configuration `[130,-70, 90, -90, -90, 0][deg]` (convert degrees to radians using the `numpy.deg2rad()` function) to verify that it indeed returns `False`.

Answer:

There are a few reasons why modeling the window with spheres is a bad idea:

A window defines a limit to our environment, like the floor, it is a border we can't cross, unlike an obstacle which we can go around. Therefore, computing spheres to avoid collision is expensive, instead of a simple condition, we will need to model enough spheres to match the wall, and check collision for each one of them. This is not only expensive, usually, spheres take some space from our environment, depending on the inflation factor. When modeling the wall with spheres we waste free space for our robot, that could be avoided if using a simple condition.

We check the wall detection for the configuration, we get:

Checking configure validity, when it hit the wall: the function return False.

We've verified it return False.

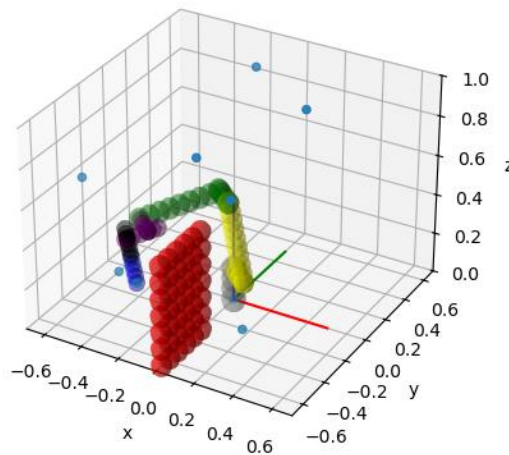
2. Run the **RRTStarPlanner** class in `RRTStarPlanner.py`.

Answer:

The result of the run:

The time it takes to compute plan is 3.379 seconds

The path cost is 3.14



3. Find paths for each test as described below. **Report** the performance in four figures, one for each **p_bias** value, plotting the cost as a function of **iteration number** for different **max_step_size** values and the success rate as a function of **iteration number** for different **max_step_size** values. Do not terminate the planner once a first solution is found but continue running it to see if and how much the path quality is improved. Discuss how the performance (cost and iteration number) of the planner is affected by the hyper-parameters **max_step_size**, **p_bias**. Since RRT* is non-deterministic you should provide statistical results (average over 20 runs for each test).¹

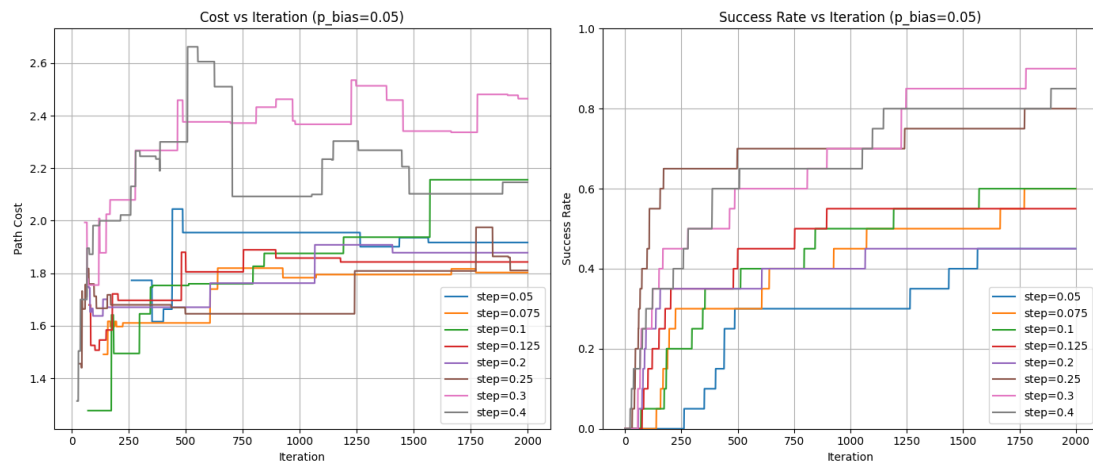
- Set **env_idx=2** in the Environment constructor.
- Limit the maximum iteration to 2000 for each run.
- Set start position to [110,-70, 90, -90, -90, 0][deg] and goal position to [50, -80, 90, -90, -90, 0][deg]

run the tests for **max_step_size** = [0.05, 0.075, 0.1, 0.125, 0.2, 0.25, 0.3, 0.4] and **p_bias** = [0.05, 0.2]

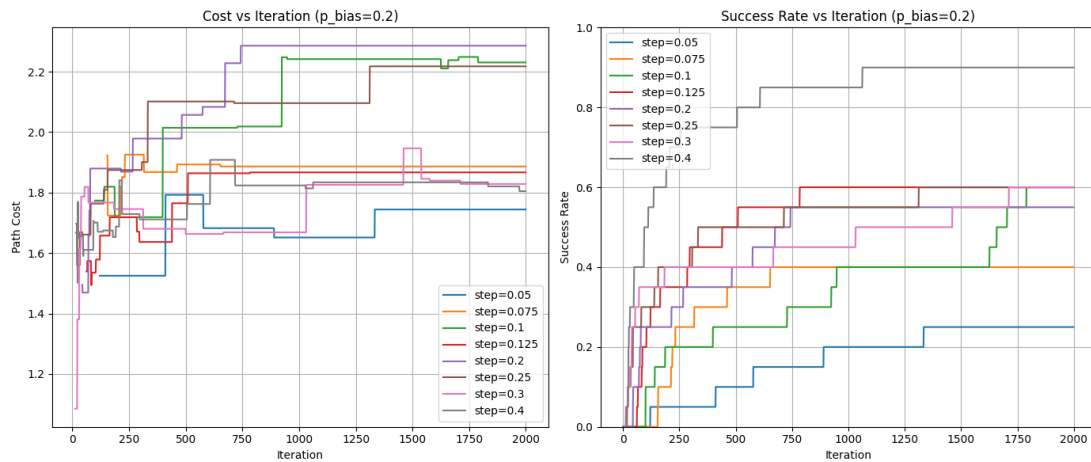
Guidance: For each combination of **max_step_size** and **p_bias**, perform 20 runs. At constant iteration intervals, record whether a solution has been obtained and its current cost. For the "Cost vs. Iteration" plots, average the cost only across the runs that have found a solution by that specific interval. For the "Success Rate" plots, report the percentage of the 20 runs that have reached a solution at each interval.

Answer:

For $p_{bias} = 0.05$:



For $p_{bias} = 0.2$:



Across all runs, we observe an early rise in average path cost, which is a statistical effect related to the Success Rate. In the initial iterations, only a few runs, those with the most favorable random samples, find a path. These paths are typically short and direct, leading to a low initial average cost. As iterations progress and the success rate rises, harder and more expensive paths start contributing to the average. This jump reflects that the algorithm goes from relying on random lucky samples to finding valid but more costly solutions.

A higher goal bias accelerates convergence and stabilizes performance. Under $p_bias=0.05$, the algorithm is exploratory. Success rates increase slowly, cost rises gradually, and small-to-medium steps produce the lowest costs. In contrast, under $p_bias=0.2$, the algorithm is greedy. Success rates increase rapidly, most step sizes achieve success before iteration 500, cost rises early but stabilizes sooner, and large steps outperform medium ones in speed and final cost. Meaning, increasing the goal bias makes the algorithm more goal directed, accelerates success, and highlight the trade-off between exploration and greedy exploitation.

Step size strongly affects path quality, but its impact change depending on the goal bias. When $p_bias=0.05$, smaller steps produce the lowest costs because the algorithm explores the space broadly, allowing rewiring to optimize paths around obstacles. Large steps generate coarser, zig zag paths that increase cost. When the goal bias is higher ($p_bias=0.2$), it changes. Large max steps (0.4) now perform well. They converge quickly and achieve low costs, because they can go over difficult regions or take shortcuts. Medium steps (0.2 and 0.1), however, sometimes become trapped in suboptimal paths due to the strong goal-directed push, leading to higher final costs (~2.2). Small steps (0.05) still yield low-cost solutions but stabilize later, when iterations are needed to fully optimize the path. Low bias favors small-to-medium steps, while high bias can make large steps more efficient in both cost and convergence speed.

4 Real World Basic Experience (10 points)

Choose the path with the lowest cost between all paths computed and bring it to the lab during the dedicated meeting (28/12 and 4/1, details to come). Together with the instructor, run it on the UR5e manipulator. include a video that visualizes it.

The best path we found has the following parameters:

$Cost = 1.0850$, $p_{bias} = 0.2$, $max_step = 0.3$

$run_index = 4$, $waypoints = 5$

The best path in degrees :

```
[[ 110.  - 70.  90.  - 90.  - 90.  0. ],  
 [ 97.45 - 81.99 67.67 - 104.56 - 103.88 - 33.15],  
 [ 76.02 - 83.48 46.66 - 114.94 - 112.69 - 21.76],  
 [ 59.92 - 81.33 73.47 - 99.51 - 98.65 - 8.3 ],  
 [ 50.  - 80.  90.  - 90.  - 90.  0. ]]
```

We saved the path in best_path3d.json.

The attached video is named:

rrt_star_3d_path_video